

How to Scale an App to 100 Million Users on GCP 🚀

#54: A Simple Guide to Scalability (7 minutes)



NEO KIM

AUG 20, 2024  PAID

Get my system design playbook for FREE on newsletter signup:

✓ Subscribed

*This isn't a sponsored post. I wrote it for someone getting started with the Google Cloud Platform (**GCP**).*

There are various ways to scale an app on the cloud, and this is just one of them. You will find references at the bottom of this page if you want to go deeper.

- [Share this post](#) & I'll send you some rewards for the referrals.

Once upon a time, there lived 2 software engineers named Dominik and James.

They worked for a tech company named Hooli.

Although they had extremely smart ideas, their manager never listened to them.

So they were sad and frustrated.



Until one afternoon when they had a wild idea to launch a startup.

And their growth rate was mind-boggling.

Yet they wanted to keep it simple. So they hosted the app on GCP.

Onward.



System Design Frontpage (Featured)

System Design Case Study

#	A	B	C	D		F	G	H
I		K	L	M	N		P	Q
R	S	T	U	V	W		Y	Z

System Design Fundamentals

#	A	B	C			G	H
				M			Q
R	S						

I built a GitHub repository to help you learn system design months ago.

I want to make it the front page for system design. And help you pass system design interviews + become good at work. Consider putting a star if you find it valuable:

Star it

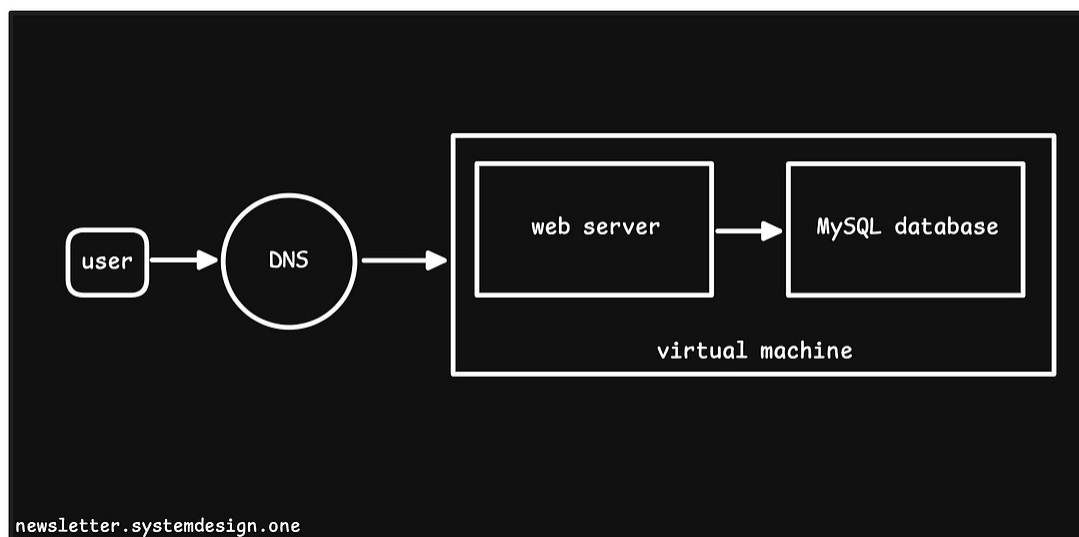
Google Cloud Scalability

Here's their scalability journey from 0 to 100 million users:

1. Thousand Users:

This is how they served the first 1000 users.

- They created a minimum viable product¹ (**MVP**) using monolith architecture - a single web server and a MySQL database.
- Then ran the app on a single virtual machine.



Minimum Viable Product Using Monolith Architecture

A virtual machine provides workload isolation and extra security.

They used Google Cloud Shell to deploy the app on GCP. It's a command-line tool for the management and deployment of apps on GCP.

While Cloud DNS routed user traffic to the virtual machine. Domain Name System (**DNS**) is a service for translating human-readable domain names into IP addresses.

Yet they had users only from North America.

So they deployed the app only in that region to keep data closer to users and save costs.

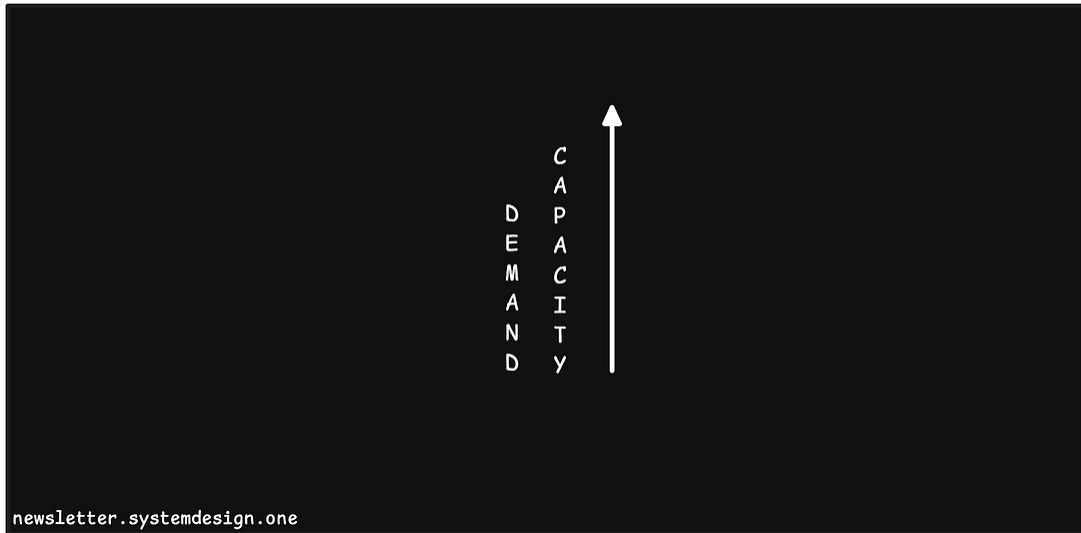
Life Was Good.

2. Five Thousand Users:

Until one day when they received a phone call as users couldn't access the app.

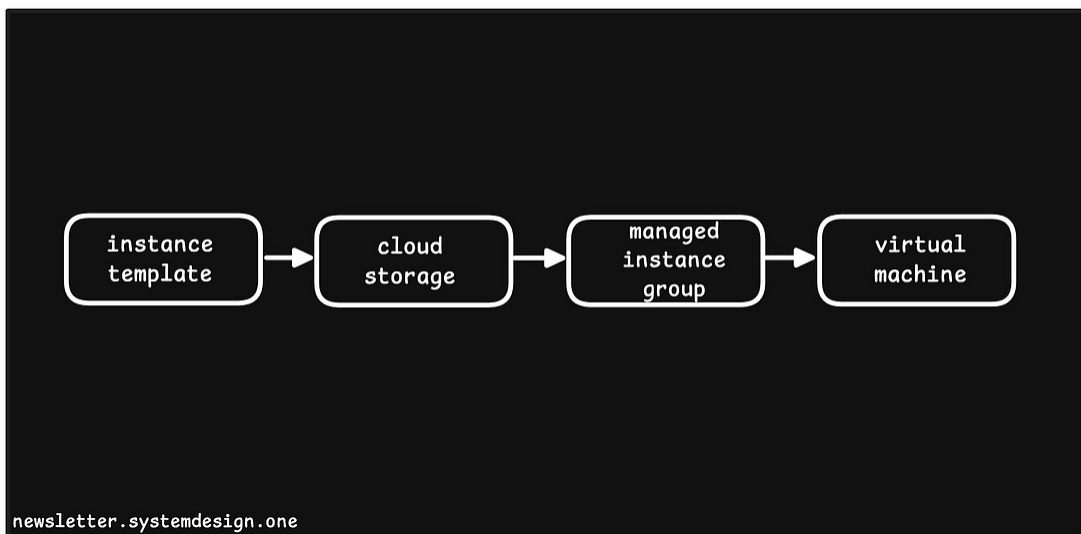
They faced performance issues due to many concurrent users. And the single virtual machine running their app crashed. (This is a single point of failure.)

While they needed a scalable² and resilient³ app.



Scaling System Capacity to Meet Demand

So they set up autoscaling⁴ using the Managed Instance Group (MIG). It uses a base instance template to create new virtual machine instances.



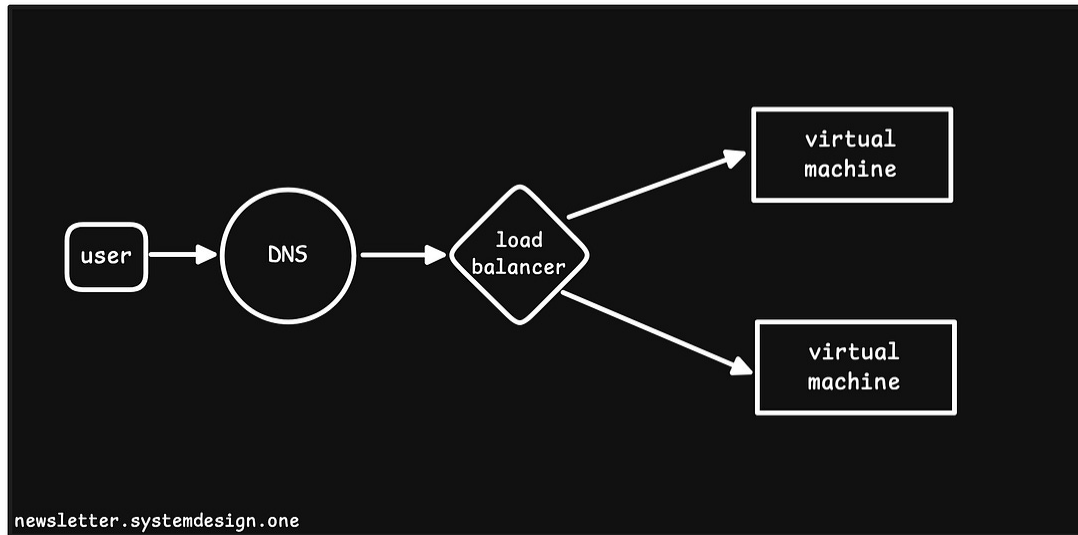
Creating a New Virtual Machine From the Instance Template

This is how they used MIG:

- Auto scaler: create and destroy virtual machines based on load (CPU & memory)

- Auto healer: recreate a virtual machine if it becomes unhealthy

And automate virtual machine installation across different zones for high availability.

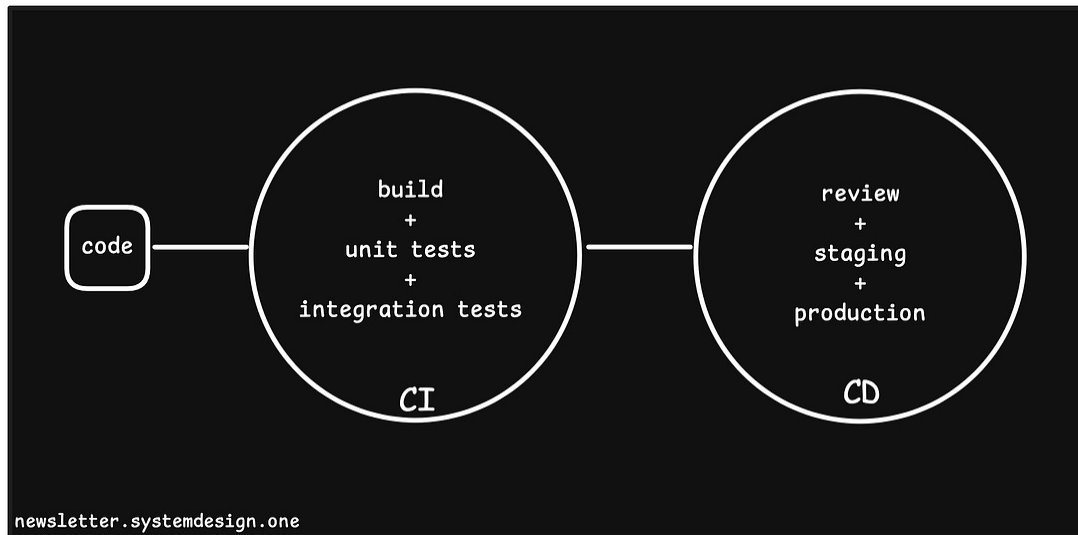


Routing Traffic Using a Load Balancer

Yet they must distribute traffic across various virtual machines for performance.

So they set up a load balancer. And route the user traffic to the single IP address of the load balancer. It then distributes the traffic across virtual machines.

Also they set up monitoring and logging to troubleshoot system failures.



CI/CD for Reliable App Releases

Besides they set up continuous integration and continuing delivery (**CI/CD**). It allowed them to automate:

- Testing
- Development
- Deployment

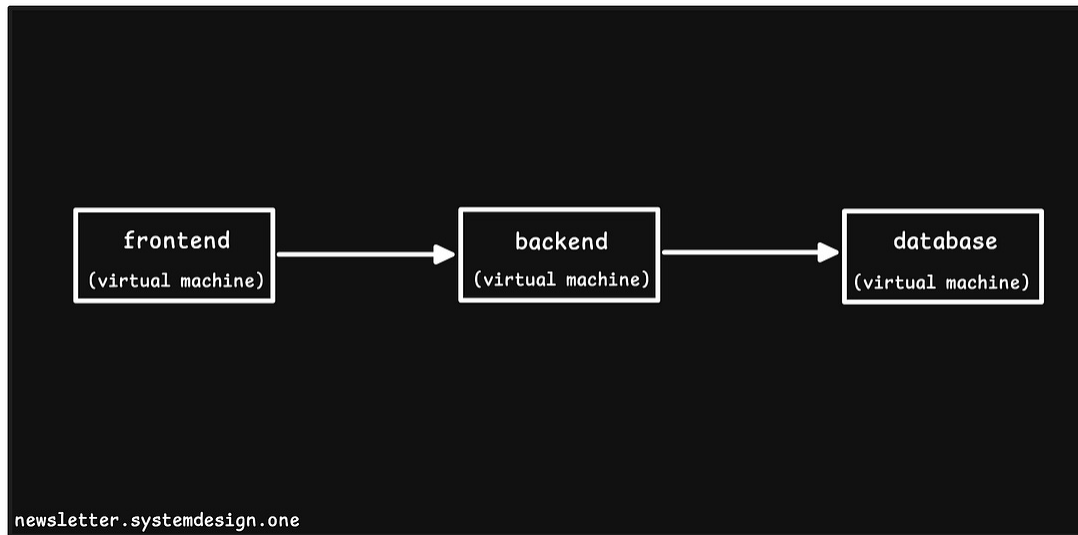
Thus get faster feedback, improve code quality, and do reliable releases.

And Life Was Good Again.

3. Ten Thousand Users:

Virtual machines failed at times.

While they ran the web server and MySQL database in the same virtual machine instance. Put simply, all functionality runs within a virtual machine. Thus limiting availability for some users on failures.



Three-Tier Architecture

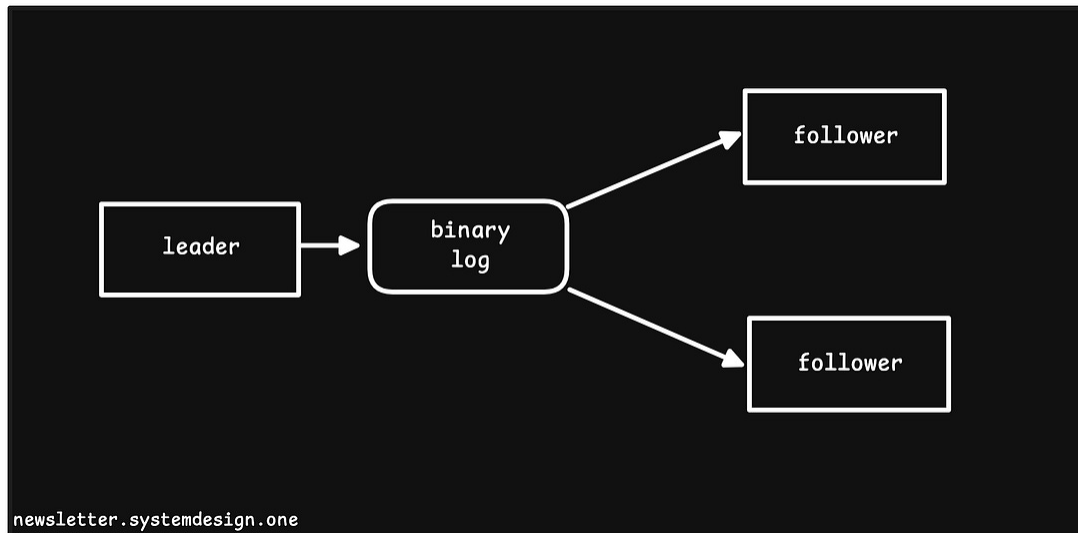
So they decoupled the app into a 3-tier architecture:

- Frontend
- Backend
- Database

And ran each layer on separate virtual machines.

They deploy virtual machines across different zones to prevent single-zone failures.

And set up auto-scaling for each layer. They use a load balancer to route traffic to the frontend layer. While another load balancer distributes traffic across the backend layer.



Leader-Follower Replication Topology in MySQL

Besides they ran MySQL in leader-follower replication topology for high availability.

Many users from Europe and Asia signed up in the meantime.

And their growth was inevitable. Yet new users faced latency issues as they were located far from the servers. So they deployed the servers across many regions on GCP.

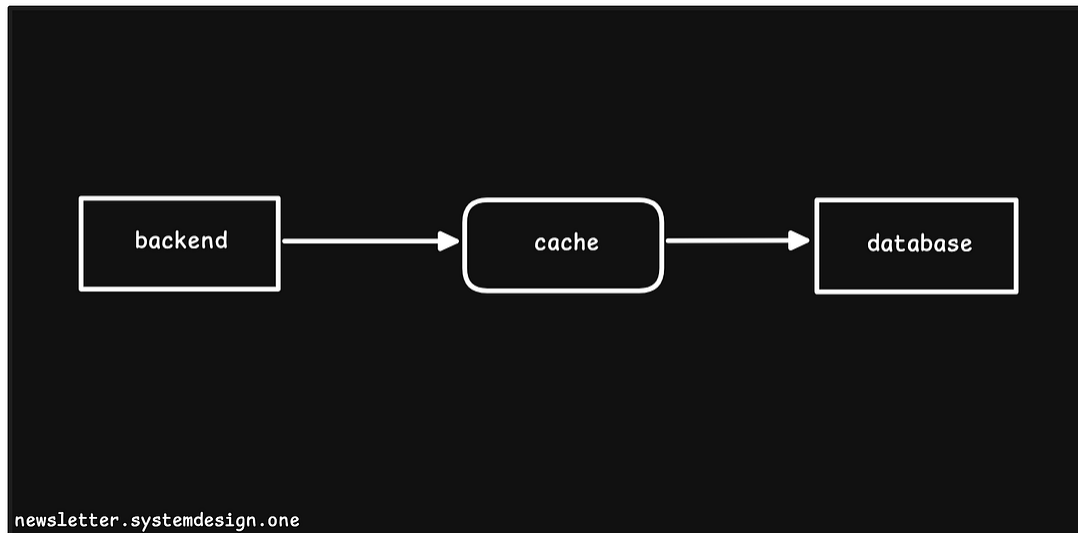
Very Neat.

4. Hundred Thousand Users:

But one day, they noticed database performance issues.

They got extreme concurrent reads and writes. And adding more disks for increased input-output operations per second (**IOPS**) needs manual operational overhead. So they moved to Google's managed relational database service.

It automatically extends the disk without downtime.



Caching Frequently Accessed Data for Fast Access

Yet the database often got queried for the same data.

So they installed an in-memory database (Redis) between the backend and database. It cached frequently accessed data for fast access. And reduced database load.

This stabilized their data layer.

Very Clean.

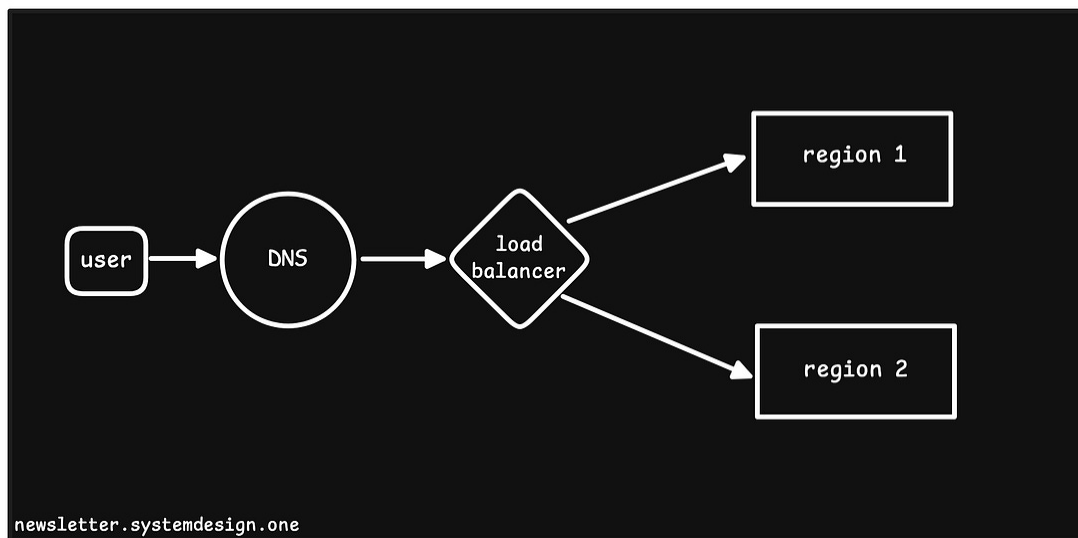
5. One Million Users:

Their growth skyrocketed.

Yet one morning, they noticed user traffic routed to failed virtual machines.

Here's what happened. They use DNS Geo routing for user traffic. But the client's DNS cache got outdated and pointed to the failed virtual machines. A simple fix is to reduce cache time to live (**TTL**). But frequent DNS policy updates are needed for better results.

So they set up the Google Cloud global load balancer.

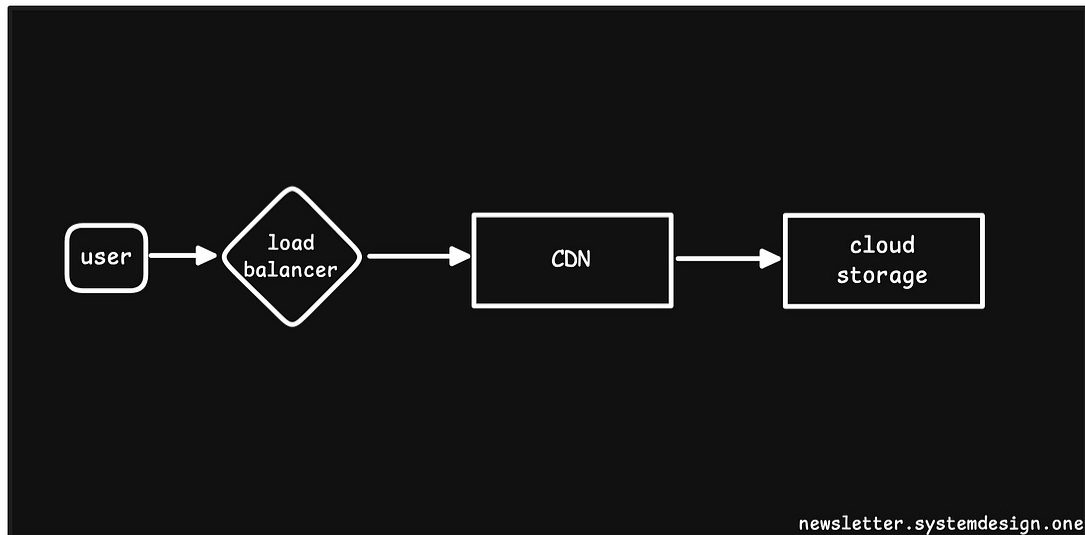


Routing Traffic via the Global Load Balancer

These are the benefits of using the global load balancer:

- A single IP address gets configured on the global load balancer, so there's no need to update the DNS policy
- It knows failed virtual machines using periodic health checks

And routes the traffic to another region if a region fails.



Serving Static Content Through CDN

Yet web pages with images were slow for some users.

So they set up the content delivery network (**CDN**) for fast delivery.
And used cloud storage for storing static content. It's an object storage.

While CDN is cheaper to serve images due to low bandwidth usage.

They wanted to scale the data layer again.

But didn't want to invest time and effort into it. (Scalability without sacrificing data consistency is difficult.)

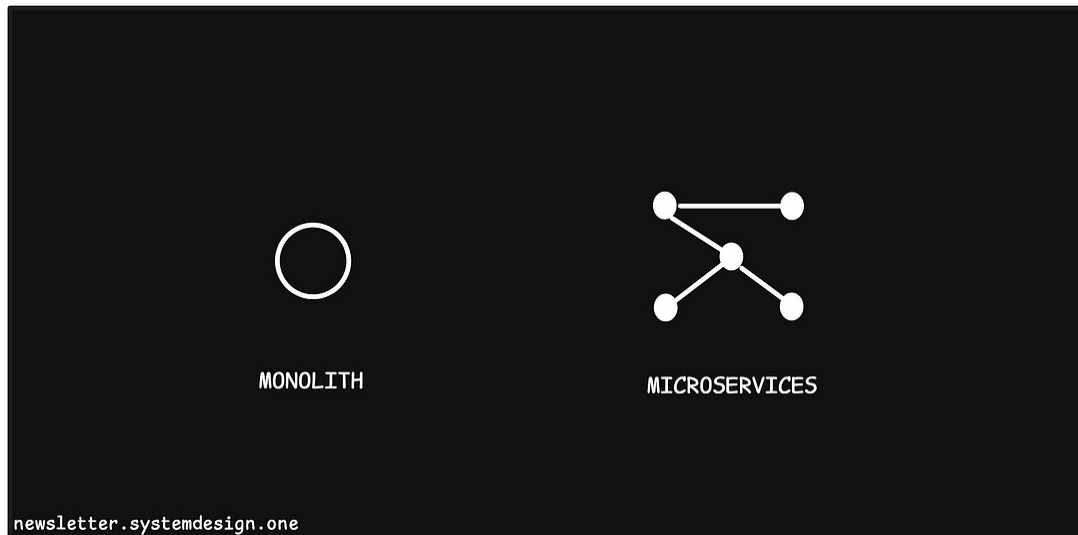
So they use Google Cloud Spanner. It's a scalable relational database with strong consistency.

Very Straight.

6. Ten Million Users:

Some features of their app became extremely popular than others.

And they wanted to scale those features separately.



Monolith vs Microservices Architecture

So they set up microservices architecture.

They split monolith to microservices based on functionality. And ran microservices in containers to ensure consistent environments across various deployment stages. It made scaling, management, and isolation of microservices easier.

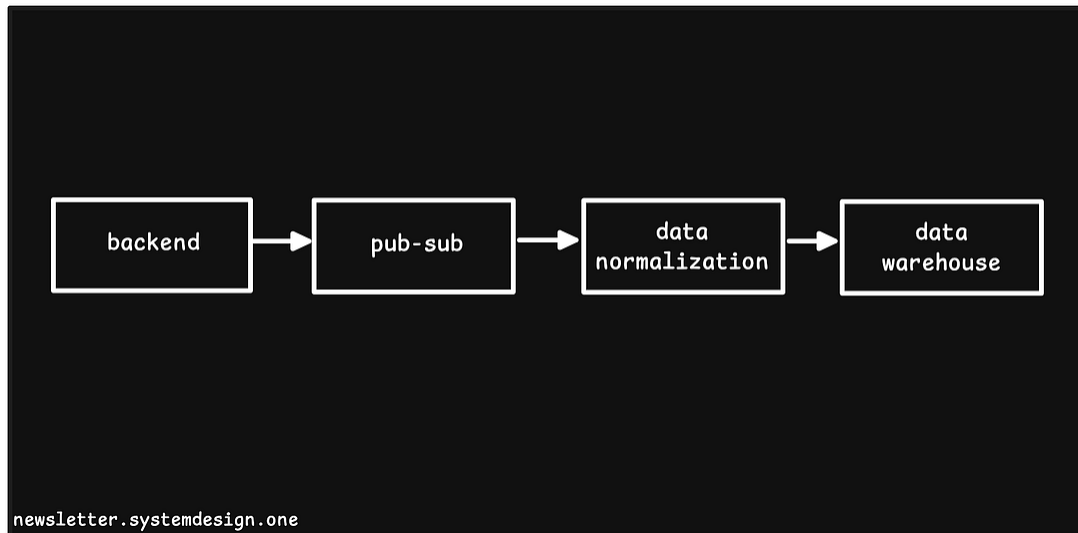
Besides they use Kubernetes to manage containers. It lets them focus on application development instead of worrying about infrastructure management.

Pretty Wild.

7. Hundred Million Users:

Their growth became exponential.

But they struggled with website optimization.



Clickstream Architecture

So they gathered clickstream⁵ data. Here's how:

- They used cloud pub-sub for asynchronous data processing. It offers scalability and guarantees at least once data delivery.
- They stored logs in cloud storage to save costs. It's an object storage.
- They used dataflow to normalize data - remove data duplicates.

Then stored processed data in BigQuery (data warehouse).

This helped them understand user behavior and get better business insights.

And they lived happily ever after.

Subscribe to get simplified case studies delivered straight to your inbox:

✓ Subscribed



Follow me on [LinkedIn](#) | [YouTube](#) | [Threads](#) | [Twitter](#) |
[Instagram](#)

Thank you for supporting this newsletter. Consider sharing this post with your friends and get rewards. Y'all are the best.

1 Referral



**Leetcode
Master
Template**

2 Referrals



**Interview
Mistakes
to Avoid PDF**

3 Referrals



**Popular
Interview
Questions PDF**

Share



How Netflix Uses Chaos Engineering to Create Resilience Systems 🐒

NEO KIM · AUGUST 5, 2024

[Read full story](#)



How Meta Achieves 99.99999999% Cache Consistency 🎯

NEO KIM · JULY 18, 2024

[Read full story](#)

References

- [Google Cloud Platform From 1 to 100 Million Users \(Cloud Next '19\)](#)
- [Regions and zones - Google Cloud](#)
- [Containers vs. virtual machines](#)
- [Three-tier vs. microservices architecture: How to choose](#)
- [What Is a Load Balancer?](#)
- [Minimum Viable Product \(MVP\) - What is it & how to start](#)
- [Spanner \(database\).](#)
- [Pattern: Health Check API](#)
- [Continuous Integration & Deployment](#)

-
- 1 **Minimum viable product** is a product with enough features to deliver customer value
 - 2 **Scalability** means the app should work properly as more users join
 - 3 **Resilience** means the app must be highly available and work properly even if a failure occurs
 - 4 **Autoscaling** is a system's ability to automatically increase or decrease

computing resources based on demand

- 5 **Clickstream** data is the information collected while a user navigates the website